

Hyperion: A Deterministic 6-DOF Flight Dynamics Engine

Simulated Flight Software Architecture for HIL Validation

C++20 · Multithreading · Orbital Mechanics

Abstract

This report details the implementation of **Hyperion**, a ground-up physics engine designed for Hardware-in-the-Loop (HIL) validation of spacecraft avionics. Unlike non-deterministic game engines, Hyperion utilizes a thread-safe, producer-consumer architecture to decouple high-frequency physics (10 kHz) from telemetry broadcasting (50 Hz). The system features a custom 4th-order Runge-Kutta integrator, a J2-perturbed gravity model, and a strictly deterministic spinlock timing mechanism to simulate Low Earth Orbit (LEO) dynamics with high fidelity.

1 System Architecture

A core requirement for flight-software validation is strict **timing determinism**. Hyperion enforces this through a multi-threaded execution model with `std::mutex`-protected coherence points and atomic state snapshots between the physics integrator and telemetry stack.

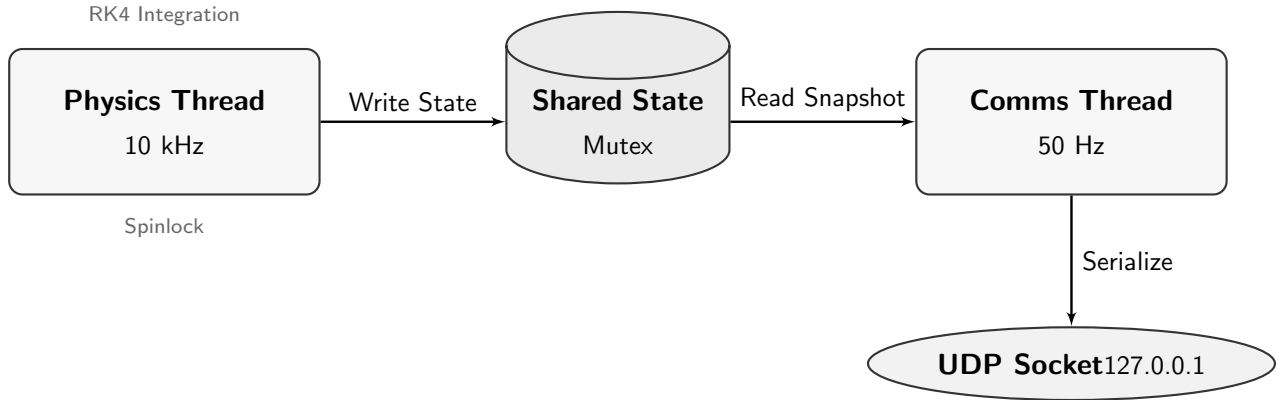


Figure 1: Hyperion concurrency model illustrating the decoupled Physics and Telemetry loops.

2 Mathematical Formulation

To eliminate dependencies on external "black box" libraries, the math kernel (Vector math, Quaternions, Integration) was implemented from first principles.

2.1 Orbital Dynamics & J2 Perturbation

For Low Earth Orbit (LEO) simulation, the standard point-mass gravity model is insufficient due to Earth's oblateness. I implemented the J_2 zonal harmonic perturbation to capture nodal precession. The total acceleration vector $\mathbf{a}$ is calculated as:

$$\mathbf{a} = -\frac{\mu}{r^3}\mathbf{r} + \mathbf{a}_{J2} \quad (1)$$

Where the perturbative terms for the J_2 effect are derived as:

$$a_{x,J2} = \frac{x}{r} \left[\frac{3}{2} J_2 \left(\frac{R_E}{r} \right)^2 \frac{\mu}{r^2} \left(\frac{5z^2}{r^2} - 1 \right) \right] \quad (2)$$

$$a_{z,J2} = \frac{z}{r} \left[\frac{3}{2} J_2 \left(\frac{R_E}{r} \right)^2 \frac{\mu}{r^2} \left(\frac{5z^2}{r^2} - 3 \right) \right] \quad (3)$$

2.2 Numerical Integration (RK4)

Time propagation is handled by a custom 4th-Order Runge-Kutta solver. This offers a significant stability improvement over Euler integration for orbital mechanics, preserving energy over longer simulation windows.

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{dt}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (4)$$

3 Implementation Details

3.1 Deterministic Timing (Spinlock)

Standard `sleep()` functions in operating systems introduce jitter ($> 1\text{ms}$), which is unacceptable for high-frequency control loops. Hyperion utilizes a busy-wait strategy (Spinlock) to strictly enforce the $100\mu\text{s}$ physics timestep.

```
1 while (running) {
2     auto start = std::chrono::high_resolution_clock::now();
3
4     // 1. Step Physics (RK4)
5     PhysicsEngine::step_rk4(local_state, PHYSICS_DT);
6
7     // 2. Thread-Safe State Update
8     {
9         std::lock_guard<std::mutex> lock(state_mutex);
10        shared_state = local_state;
11    }
12
13    // 3. Spinlock (Busy Wait) for Determinism
14    while (std::chrono::duration<double>(
15        std::chrono::high_resolution_clock::now() - start).count() < PHYSICS_DT
16    ) {
17        // Burn cycles to ensure exact 10kHz frequency
18    }
```

Listing 1: High-Precision Timing Loop

4 Verification & Visualization

To validate the physics engine, a Python-based telemetry dashboard was developed to listen to the UDP stream in real-time.

4.1 Energy Conservation Check

Since drag is currently neglected, the total Specific Mechanical Energy ($\mathcal{E}$) of the orbit must remain constant.

$$\mathcal{E} = \frac{v^2}{2} - \frac{\mu}{r} = \text{constant} \quad (5)$$

The telemetry data confirms integrator stability, with energy deviations $< 10^{-5} \text{ km}^2/\text{s}^2$ over the simulation window.

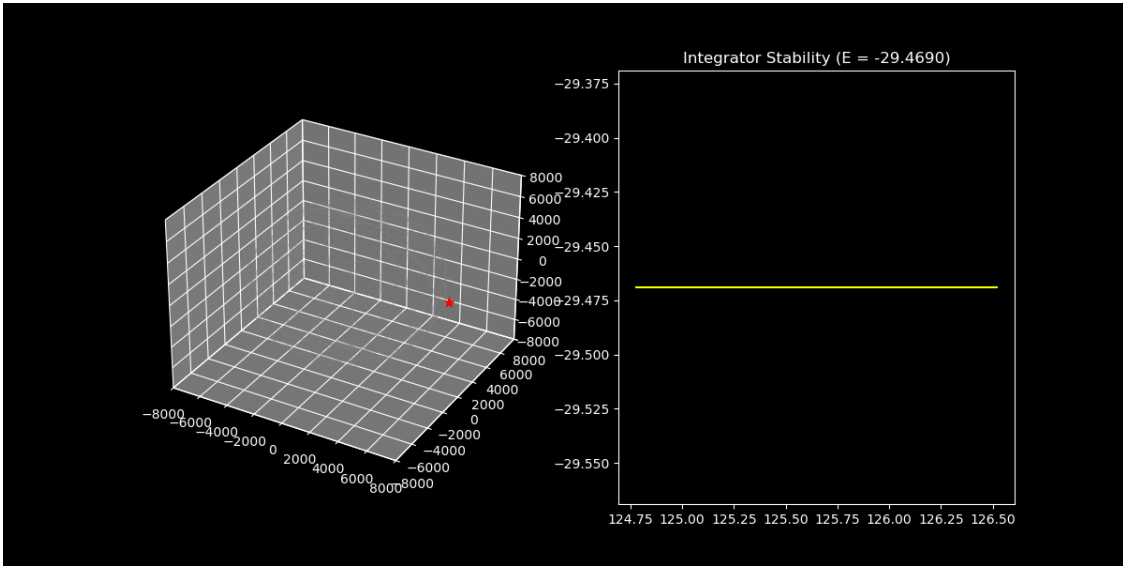


Figure 2: Real-time telemetry dashboard showing 3D ECEF Orbit trajectory (Left) and Specific Mechanical Energy stability plot (Right).

5 Future Work

Future iterations will introduce a quaternion-based error state Kalman Filter (ESKF) to process noisy IMU data, closing the loop for full Guidance, Navigation, and Control (GNC) simulations.